Constructing Optimal Bushy Join Trees by Solving QUBO Problems on Quantum Hardware and Simulators

Nitin Nayak*
Jan Rehfeld*
nitin.nayak@uni-luebeck.de
Jan_Rehfeld@gmx.de
Universität zu Lübeck
Lübeck, Germany

Tobias Winker
Benjamin Warnke
Umut Çalıkyılmaz
Sven Groppe
t.winker@uni-luebeck.de
benjamin.warnke@uni-luebeck.de
umut.calikyilmaz@uni-luebeck.de
sven.groppe@uni-luebeck.de
Universität zu Lübeck
Lübeck, Germany

ABSTRACT

The join order is one of the most important factors that impact the speed of query processing. Its optimization is known to be NP-hard, such that it is worth investigating the benefits of utilizing quantum computers for optimizing join orders. Hence in this paper, we propose to model the join order problem as quadratic unconstrained binary optimization (QUBO) problem to be solved with various quantum approaches like QAOA, VQE, simulated annealing and quantum annealing. In this paper, we will show the enhanced performance using our QUBO formulation to find valid and optimal shots for real-world queries joining three, four and five relations on D-Wave quantum annealer. The highlight of this study will demonstrate that we were able to generate valid and optimal join orders for five relations using the D-Wave quantum annealer, with values of about 12% and 0.27% respectively and we were able to find the optimal bushy join trees. Experiments for the gate-based simulations were able to produce optimized join order for three and four relations including bushy trees using QAOA and VQE algorithms.

KEYWORDS

Query Optimization, Join-Ordering, VQE, QAOA, Quantum Simulation, and Quantum Annealing.

ACM Reference Format:
Nitin Nayak, Jan Rehfeld, Tobias Winker, Benjamin Warnke, Umut Çalıkyılmaz, and Sven Groppe. 2023. Constructing Optimal Bushy Join Trees by Solving QUBO Problems on Quantum Hardware and Simulators. In *The International Workshop on Big Data in Emergent Distributed Environments (BiDEDE '23), June 18, 2023, Seattle, WA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3579142.3594298

*Both authors contributed equally to this research.

BiDEDE '23, June 18, 2023, Seattle, WA, USA

ACM ISBN 979-8-4007-0093-4/23/06…$15.00
https://doi.org/10.1145/3579142.3594298

1 INTRODUCTION

Choosing the optimal query plan during query optimization minimizes the cost of query processing in Database Management Systems (DBMS). Join order optimization (JOO) is one of the most crucial tasks of query optimization. However, constructing optimal bushy join tress is known to be NP-hard [35], which might benefit from accelerating the optimization problem using quantum computers [4]. In this paper, we propose to formulate the JOO problem as quadratic unconstrained binary optimization (QUBO) problem, which can be solved on quantum annealers [23] and gate-based quantum computers using approaches like the variational quantum eigensolver (VQE) [9] and the quantum approximation optimization algorithm (QAOA) [15]. In contrast to a previous approach [37] to find the best solution among the left-deep join trees by solving QUBO problems, our approach supports the more general bushy join trees and hence finds the optimal solution among all possible join trees[1].

Our main contributions are:

(1) Optimizing the join order by solving a QUBO problem supporting also bushy join trees,
(2) a comprehensive evaluation using real-world queries of applying various approaches like simulated annealing, quantum annealing, VQE and QAOA on quantum hardware and simulators, and
(3) an experimental comparison of the number of valid and optimal join trees up to 7 relations for real-world queries using simulated annealing and the various mentioned quantum approaches.

In Section 2, we introduce the fundamentals of quantum mechanics and quantum computing, with a focus on the methods for solving QUBO problems. We define the problem of join order optimization and describe how to model it as QUBO problem in Section 3. We describe the experimental evaluation of approaches like simulated annealing, quantum annealing, VQE and QAOA on quantum hardware and simulators in Section 4. Finally, we conclude by summarizing the main ideas of the paper in Section 5.

[1]with reference to cost estimations like cardinality or I/O cost estimations

2 QUANTUM COMPUTING FUNDAMENTALS

Quantum Computing exploits the fundamentals of physics like superposition, entanglement and interference to do the computing and these physical phenomena are responsible for quantum computing to be very different from classical computing. In the following sections, we will discuss these important basics.

2.1 Quantum Mechanics and Quantum Information

As classical physics has been proven to be inadequate for explaining the behaviour of very small particles, quantum mechanics was developed at the beginning of the 20th century to achieve this purpose [16, 34]. Although its prepositions do not comply with intuition, its predictions have been used to prove many physical phenomena over the decades [7, 21, 32]. It has revolutionized various scientific fields, such as chemistry, materials science and information theory.

In the most basic sense, the postulates of quantum mechanics propose to represent a physical system by a vector of probability amplitudes, instead of a deterministic state. As a result, the outcome of a measurement on a quantum system is uncertain. Assume a physical system that gives one of the two outcomes, $|0\rangle$ or $|1\rangle$, when measured. If it is a quantum system, it can be in a state $|\psi\rangle$ such that

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \tag{1}$$

where α and β are two complex numbers with property $|\alpha|^2+|\beta|^2=1$. Then it is said that the state of the system is a superposition of $|0\rangle$ and $|1\rangle$.

When such a system is measured, it assumes the final state $|0\rangle$ with probability $|\alpha|^2$ and $|1\rangle$ with probability $|\beta|^2$ [34]. In quantum mechanics, measurement is an irreversible process, that erases the previous information on the system and makes it *collapse* into one of the possible outcomes (the basis states of the measurement operator). Apart from measurement, the evolution of a quantum system is always a reversible process, which is represented by a unitary operator in the formalism of quantum mechanics.

In the field of quantum information, a two-state quantum system is called a qubit [29]. A qubit is the most basic unit of quantum information, so in a sense, it is the quantum counterpart of a classical bit. The main difference from its classical counterpart is that it can be in a superposition of the basis states, such as in (1). Another important difference is the ability of a system of multiple qubits to be in an *entangled* state. There is an example of such a state below

$$|\psi\rangle = \alpha|00\rangle + \beta|11\rangle. \tag{2}$$

When the qubits are measured the outcome is either $|00\rangle$ or $|11\rangle$. There is no possibility of measuring one qubit as $|0\rangle$ and the other as $|1\rangle$, so the outcomes of the qubits are correlated. This feature is one of the reasons for the superiority of quantum computing.

2.2 Quantum Computing

The Intent of quantum computing is to process quantum information in order to gain an advantage on some computational tasks, which are not tractable using a classical computer [12]. In the last few decades, the superiority of quantum computing over classical ones are proven over some problems. This superiority is granted by using specially designed quantum algorithms such as Grover search [17, 18] and Shor's algorithm for factoring [39].

There are different paradigms for quantum computing. In gate-based quantum computing (also called quantum circuit model), algorithms are designed as circuits where quantum gates are placed on strings of qubits [29]. In this sense, it is similar to classical computing. Quantum gates are unitary quantum operations that make the qubits evolve in a desired way. At the end of the circuit, the qubits are measured to obtain classical information from them. Grover's and Shor's algorithms are both designed to work on a gate-based quantum computer.

Adiabatic quantum computing works on a different principle than gate-based computation [2]. The problem at hand is formulated as a Hamiltonian in a way that the solution to the problem is the ground state of this Hamiltonian. The initial Hamiltonian is set to a known function with an easy-to-prepare ground state, and the initial state of the qubits is set to the ground state of it. If the energy of the system is altered slowly enough from the initial Hamiltonian to the final one, the final state would be the solution state according to the adiabatic principle.

2.3 QUBO Problems

Quadratic unconstrained binary optimization (QUBO) problems are problems with binary decision variables, quadratic objective function and no hard constraint [14]. QUBO formulation is important for quantum optimization since problems that are formulated this way can be represented by an Ising Hamiltonian [19]. Well known optimization problems, such as travelling salesman problem, can be reformulated as QUBO problem [30]. This would allow the problem to be solved using certain quantum algorithms. In this paper, we aim to follow a similar path for join order optimization problem.

As there are different quantum computing paradigms, there are also different methods to solve QUBO problems by quantum computing. The common point in all these approaches is that the problem is translated to a Hamiltonian of Ising model, such that the optimal solution is the ground state of it. Then, an initial state is set and slowly evolved to be close to the optimal solution state by either applying quantum gates or altering the magnitude of the field acting on the state.

2.3.1 Quantum Annealing. In classical computing, simulated annealing is the heuristic optimization approach that emulates the metallurgic process of annealing [20]. In this approach, a parameter called temperature determines the probability of deviating to a higher cost point to avoid local minima. This parameter is decreased slowly with the purpose of finding the optimal point, or at least a near-optimal point, at the end. If the parameter is decreased slow enough, finding the optimal point is guaranteed [5].

Quantum annealing follows a similar approach by slowly altering the field eacting on a system of qubits [13]. It runs on specially designed quantum computers called quantum annealers, which are somewhat similar to adiabatic quantum computers [28]. At the beginning, the magnitude of the transverse field is at maximum, and the initial state is set to its ground state. The varying field strength is prepared in a way that when the transverse field become zero, the Hamiltonian of the system of qubits is the Hamiltonian

representation of the QUBO problem. Thus, a sufficiently slow evolution of the field will result in an optimal or near-optimal point.

2.3.2 Variational Quantum Algorithms. A variational quantum circuit (VQC) is a quantum circuit with parameterized quantum gates that can be changed for each run. Variational quantum algorithms (VQA) employs a VQC and tries to find its optimal parameter values by a classical optimization routine [8]. The main purpose of these algorithms is to share the workload between quantum and classical computers not to exceed the hardware limitations of quantum computers [33].

Quantum approximate optimization algorithm (QAOA) is a VQA that is used to solve optimization problems [11]. This algorithm is inspired by quantum annealing and its process is similar to a discrete time approximation of QA. Variational quantum eigensolver (VQE) is another VQA that is used to find the eigenvalues of a given Hamiltonian [31]. It is used as an optimization technique, since it can find the eigenvaule of the ground state, wich corresponds to the optimal value of a QUBO problem. VQAs are designed to work on gate-based quantum computers.

2.4 Further Related Work

Because of the importance of the problem, there have been various approaches to solve join order optimization over the years. These include dynamic programming [38], mixed integer programming [22], genetic algorithm [40], ant colony optimization [24], particle swarm optimization [27], simulated annealing [41] and machine learning [25] methods. We suggest the reader to refer to these works for a better understanding of the problem.

Quantum approaches for join ordering also exist. In [37], a QUBO formulation and quantum annealing solution to the problem is discussed. However, the approach in [37] is restricted to left-deep join trees while our solution determines optimal bushy join trees, i.e., the optimal join tree among all possible ones. A quantum annealing approach for the problem of multiple query optimization is introduced in [42]. The authors of [43, 44] introduce the first approaches of using quantum machine learning [6, 36] for join ordering. We also would like to direct the reader to other quantum optimization approaches that could be used for join ordering, such as quantum exact optimization [10], filtering VQA [3] and variational imaginary time evolution [26].

3 QUBO FORMULATION

In this section, we define a QUBO problem to model the challenge of determining the optimal join order in a combinatorial optimization setting. We can further use a quantum computer or simulator to solve this QUBO problem and finally retrieve the optimized solution.

3.1 Problem Definition

We assume that the relations $R_1, ..., R_m$ are to be joined in a given query and define the relation set $M := \{R_1, ..., R_m\}$ with $|M| = m$. We consider only binary joins in this paper, i.e., each join has exactly two operands. The power set P of all elements in M then contains all possible symmetric combinations of the relations among themselves. For example, the power set for three relations contains all possible join orders, i.e., $P = \{\{\}, \{R_1\}, \{R_2\}, \{R_3\}, \{R_1, R_2\}, \{R_1, R_3\}, \{R_2, R_3\}, \{R_1, R_2, R_3\}\}$. A join of several relations is then modeled as a subset of P, e.g., $(R_1 \bowtie R_2) \bowtie R_3$ is modeled as $\{\{R_1\}, \{R_2\}, \{R_3\}, \{R_1, R_2\}, \{R_1, R_2, R_3\}\}$. Not all subsets of P are valid join combinations (see Section 3.2). As the single relations are always included in the join, we can remove them for decreasing the number of variables in the power set and hence the number of qubits for the quantum device. The same applies to the empty set[2]. In this way, $(R_1 \bowtie R_2) \bowtie R_3$ is modeled as $\{\{R_1, R_2\}, \{R_1, R_2, R_3\}\}$.

For each element in P exists a certain weight $w_i \in \mathbb{R}$ representing the cost of the respective single join. Due to simplicity, we assume symmetry for all relations and their joins, i.e., the join $R_1 \bowtie R_2$ has the same cost as the join $R_2 \bowtie R_1$. The join order of relations that globally causes the lowest costs is sought.

3.2 Modeling of the QUBO problem

In the following formulas, we notate P_k as set of elements representing joins of k relations, i.e., $P_k = \{a | a \in P \wedge |a| = k\} \subseteq P$.

The elements in P represent joins that possess respective weights (i.e., processing costs[3]) which are given as w_i for all single joins of relations $i \in P$. Binary variables x_i represent then the occurrence of a single join (i.e., an element in P) in the entire join to be processed. The QUBO formula presented below can be broken down into several self-contained subtotals, which can be understood as individual QUBO problems. In order to generally prefer joins with low costs, the first step is to subtract all weights w_i from a constant w_{max} for the maximum costs. In doing so, w_{max} should be set to be higher than the greatest weight within the current instance of the problem. This can be calculated by adding the constant $c > 0$ to the highest of all weights w_i, i.e., $w_{max} = max(w_i) + c$.

The maximum weight w_{max} serves to keep the weight of the individual relations and joins as low as a possible reward by subtracting it from each w_i. Low weights thus receive a value that is particularly far in the negative range. These are then preferred during optimization. This affects all weights of the joins in P:

$$S_1 = \sum_{j=2}^{m} \sum_{i \in P_j} x_i * (w_i - w_{max}) \tag{3}$$

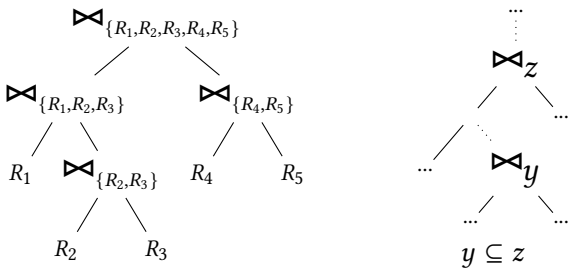
The sum S_1 thus forms the basis of the QUBO formula, for which we want to find the minimum. Since all weights in S_1 are negative, each of the binary variables would be assigned a value of 1 in a solution of S_1 resulting in a join containing all possible single joins, which is not an allowed join combination.

In the next step, constraints are defined that limit the result set to exclusively allowed join combinations, which we also call valid join trees. This can be achieved by multiplying the w_{max} weight as penalty weight to the binary variables of such combinations representing invalid join trees.

Looking at examples of valid join trees (see Figure 1a) we recognize that the set z of joined relations of a join $\bowtie_z$ subsumes the joined relations y of any join $\bowtie_y$ in arbitrary depth below $\bowtie_z$ in a valid join tree (see Figure 1b). Furthermore, a relation R_i appears

[2] and also to the variable representing the last join between all relations (e.g., $\{R_1, R_2, R_3\}$ in the example), which we do not consider in this paper and leave removing the variable of the last join as future work.

[3] Very often the number of join results, or another metric that is based on the number of join results, is used for estimating the costs in database management systems (e.g., PostgreSQL).



(a) Example **(b) Constraint**

Figure 1: Example and constraint for valid join trees

only once in a valid join tree in a leaf node, and all relations in z are leaf nodes in the subtree of $\bowtie_z$ (in arbitrary depth). We have to punish all other combinations, i.e., if y and z have common relations ($y \cap z \neq \emptyset$), then we have to punish those combinations, where the larger or equal-sized set z does not subsume y (i.e., $y \not\subseteq z$):

$$S_2 = \sum_{i=2}^{m-1} \sum_{j=i}^{m} \sum_{\substack{y \in P_i \\ \wedge z \in P_j \\ \wedge y \cap z \neq \emptyset \\ \wedge y \not\subseteq z}} x_y * x_z * w_{max} \tag{4}$$

The global minima of S_2 contain all valid join trees. Overall, we have to find the minimum of $S = S_1 + S_2$ to retrieve the optimal valid join tree with the smallest costs.

3.3 Example

In this section, we present an example of the QUBO formula for optimal join orders for the three input relations R_1, R_2 and R_3: The possible join orders for these relations are included in the set $P = \{\{R_1, R_2\}, \{R_1, R_3\}, \{R_2, R_3\}, \{R_1, R_2, R_3\}\}$. The set $\{R_1, R_2, R_3\}$ is the combination of $\{R_1, R_2\} \bowtie R_3$, $\{R_1, R_3\} \bowtie R_2$ or $\{R_2, R_3\} \bowtie R_1$ and is always included in the solution set.

In this example, we use the weights $w_{\{R_1,R_2\}} = 2$, $w_{\{R_1,R_3\}} = 4$, $w_{\{R_2,R_3\}} = 7$ and $w_{\{R_2,R_2,R_3\}} = 5$. We use the maximum weight $w_{max} = max(weights) * 2 = 7 * 2 = 14$ in this example.

Binary variables x_i represent the joins of each of the four sets of relations: $\{R_1, R_2\} \rightarrow x_0$, $\{R_1, R_3\} \rightarrow x_1$, $\{R_2, R_3\} \rightarrow x_2$ and $\{R_1, R_2, R_3\} \rightarrow x_3$. Based on this information, the target function $f(x)$ is then generated by the combinatorial constraints as presented in Section 3.2.

$$f(x) = min\left(-12x_0 - 10x_1 - 7x_2 - 9x_3 + \frac{112x_0x_1 + 112x_0x_2 + 112x_1x_2}{2}\right) \tag{5}$$

The front, linear part of the function consists of the respective weight $(w_i - w_{max}) * x_i$. In the quadratic part of the function, the binary variables of the forbidden join combinations are multiplied by a penalty weight: $x_0 * x_1 \rightarrow ((R_1 \bowtie R_2) \bowtie (R_1 \bowtie R_3))$, $x_0 * x_2 \rightarrow ((R_1 \bowtie R_2) \bowtie (R_2 \bowtie R_3))$ and $x_1 * x_2 \rightarrow ((R_1 \bowtie R_3) \bowtie (R_2 \bowtie R_3))$.

In this case, the minimum is achieved with the following variable allocation: $x_0 = 1$, $x_1 = 0$, $x_2 = 0$, and $x_3 = 1$. This corresponds to the two single joins $\{R_1, R_2\}$ and $\{R_1, R_2, R_3\}$ and thus to the join order $(R_1 \bowtie R_2) \bowtie R_3$.

Table 1: Number of binary variables/qubits

#relations:	3	4	5	6	7	8	9	10
#qubits:	4	11	26	57	120	247	502	1013

3.4 Number of Qubits

Considering the problem definition in Section 3.1 and the model of our QUBO problem for solving join order optimization in Section 3.2, we need a binary variable for each possible subset of the relations (with more than 1 relation). Hence, the number of binary variables in the QUBO formulas in our approach is $2^m - m - 1$. The number of binary variables corresponds to the number of (logical) qubits for applying quantum annealing as well as VQE and QAOA. Although the growth in the number of qubits is exponential (see Table 1), the number of supported qubits of today's quantum annealers (like D-Wave's flagship quantum annealer supporting 5,627 qubits) is far away from the number of relations used in practice.

4 EXPERIMENTAL EVALUATION

We describe the experimental environment in Section 4.1 before we analyze the results of various approaches using simulation and quantum computers for real-world queries in Section 4.2.

4.1 Experimental environment

For optimization using annealing, we have used D-Wave's simulator applying simulated annealing and D-Wave's Sampler to run the query on a real quantum annealer. We have used the Dwave annealing solver Advantage-system 4.1. It supports 5,627 qubits and 15-way qubit connectivity working at $15.4 \pm 1mK$. We have used dwave-neal==0.5.9 for simulated annealing.

Furthermore, we have used the qiskit version 0.41.1, qiskit-terra 0.23.1, qiskit-optimization 0.5.0 and qiskit-aer 0.11.2 for simulating quantum approaches on gate-based quantum computers.

The MacBook Air 2020, which has an Apple M1 processor configuration and 8GB RAM, was the platform for our experiments.

For our experiments, we use queries for the ErgastF1[4] dataset, which contains information about the Formula One series. The size of the ErgastF1 dataset is large enough so that different join orders have a significant difference in costs. The dataset consists of 13 tables and 19 foreign key relations between them. We create queries between those tables which can be joined without requiring a cross join. Our queries contain the primary key-foreign key relations as join conditions, but no other filters and we use SELECT * to return all results.

We use PostgreSQL (v. 14.4) to calculate the weights required for our QUBO model. We choose the cardinalities predicted by the internal optimizer of PostgreSQL as our weights, which are also the basis for cost estimations in the PostgreSQL optimizers as well. The estimated cardinality of a join is calculated from the estimated cardinalities of the two joined tables and the estimated selectivity of the join conditions [1]. To calculate these values, PostgreSQL stores the estimated sizes of tables and the number of distinct values, most common values, and histograms for value distributions for each column.

[4] http://ergast.com/mrd/

Table 2: Table for Percentage of Valid and Optimal Shots

	D-Wave Simulator Shots		D-Wave Annealer Shots	
Relations	Valid	Optimal	Valid	Optimal
3	99.996%	98.27%	69.788%	66.06%
4	58.95%	23.36%	37.172%	6.692%
5	32.05%	5.116%	12.008%	0.268%
6	15.147%	0.033%	0.742%	0.0%
7	9.537%	0.002%	NA	NA
8	5.96%	0.0%	NA	NA

Table 3: Table for time required by the D-Wave Simulator

Relations	#Queries	Total time (sec.)	Avg. time (sec.)
3	66	23.35	0.35
4	185	340.61	1.84
5	50	249.87	5
6	50	680.28	13.6
7	25	985.79	39.43
8	21	2712.4	129.16

4.2 Experiments with real-world queries

In this section, we will examine the experiments running the join order optimization of the real-world queries on the D-Wave simulator, D-Wave annealer, QAOA and VQE algorithms. We will also analyze the time taken for the experiments. Our QUBO problem model appears to be correctly generating the optimal and valid join order tree for three relations almost 100% of the time for the benchmark we used, as we can see in Table 2 for D-Wave simulator. While it is also true for the QAOA and VQE algorithms for our used real-world queries, we have tested all possible queries for three and four relations. For more than four relations, we have tested more than 20 queries. In comparison to the current work for the join order optimization done in the paper [37], referring to Table 3, we can see that with our QUBO model, the quantum annealer of D-Wave has better performance for each number of relation till 5 relations for 1000 shots. Three relations using real-world queries have almost twice of valid shots and six times of optimal shots. Using D-Wave quantum annealer, we were also able to obtain 12.008% of valid shots and 0.27% of the optimum shots for queries joining 5 relations out of 1000 shots. D-Wave's quantum annealer was able to produce 0.742% valid shots and none of the optimal shots for the queries joining 6 relations. Considering D-Wave's simulator, we find the results for small-scale queries very promising for the 10000 shots. We were also able to get the optimal and valid solutions for the queries joining up to seven relations while only valid shots for the queries joining 8 relations.

Execution time for the each query (QPU access time) on D-Wave annealer, which we call T can be broken down like this:

$$T = T_s + \Delta + T_p \tag{6}$$

where T_s is the sampling time, T_p is the programming time and Δ (reported as qpu access overhead time by SAPI (Solver API) and not included in the qpu access time SAPI field that reports the QPU-usage time being charged). We observed that the T_p is constant for each query (for each number of relations), which is around 15.758ms and Δ was varying between 0.5 to 2ms. Sampling time per sample can be further broken down as

$$T_s/R = T_a + T_r + T_d \tag{7}$$

R stands for the number of reads. We found that $T_a = 20\mu s$ which is single-sample annealing time and $T_d = 21\mu s$ which is single-sample delay time for each query. So the variation in the sampling time per sample was majorly decided by the T_r (single-sample readout time). For the queries joining three relations, the average value for the readout time per sample was 50μs, 71.2μs for queries joining four relations and 82μs for five relations. The average QPU access time (see eq. (6)) of queries joining three relations for 1000 number of reads was 106.758ms, and similarly for a higher number of relations.

D-Wave Simulator execution time for each query with 10000 shots for three relations is about 0.35 seconds (see table 3). Keeping the shots constant the execution time required increased with the number of relations of the query.

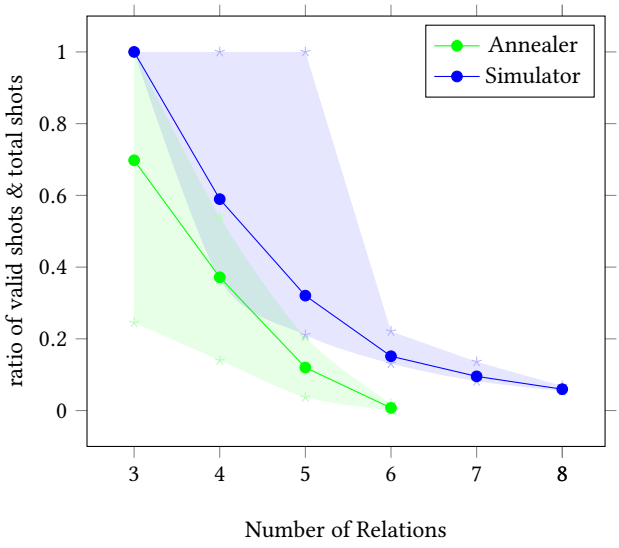


Figure 2: Ratio of Valid shots (minimum, mean and maximum) for real-world queries

We discovered some truly remarkable findings when we analyzed the valid shots acquired for each real-world query of each number of relations on the D-Wave simulator and D-Wave annealer. Even for five relations in the fig. 2, some queries had around 100% of valid shots in the case of the simulator, and it is around 20% in the case of the D-Wave annealer. In the case of the simulator, we were able to find valid shots for almost all queries for 3 relations with 100% accuracy. Maximum valid shots are even very close to 100% for some queries joining four and five relations for D-Wave simulator. Queries joining eight relations are having consistent outcomes around 5.96% for each query. In the meantime, D-Wave annealer generated valid join trees for queries with at most six relations. Accordingly, the ratio of the maximum optimal shots

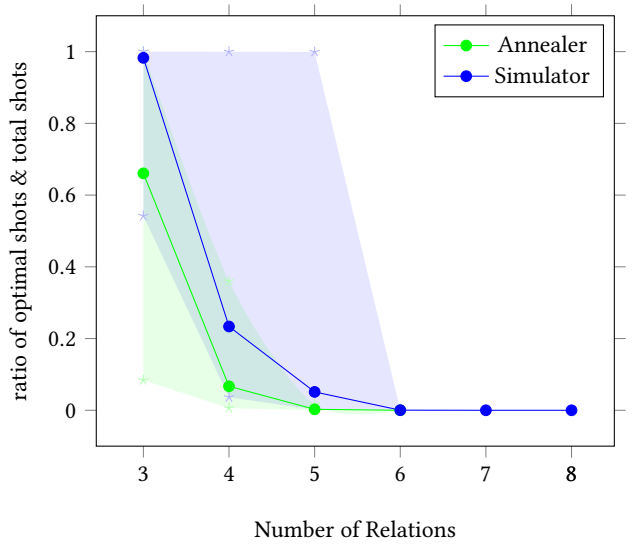


Figure 3: Ratio of Optimal shots (minimum, mean and maximum) for the real-world queries

looks very similar to valid shots on the D-Wave simulator up to five relations. Beyond five relations the simulator has a consistent number of optimal shots and it goes to zero for eight relations. Surprisingly, a few queries joining three relations have 100% of optimal shots on D-Wave annealer and around 36% of optimal shots for queries with four relations.

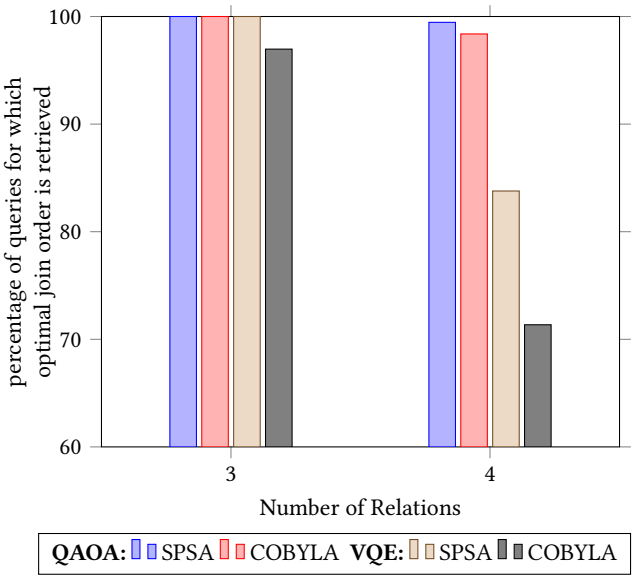


Figure 4: Real-world queries on Qiskit simulator using QAOA and VQE algorithms with SPSA and COBYLA optimizers

In the fig. 4, for the gate-based quantum simulator we have applied the QAOA (reps=1) and VQE (ansatz=TwoLocal, reps=1) algorithms with linear entanglement for our QUBO formulation, although the performances of only a small number of relations have been demonstrated by gate-based simulators. In our case, we could only optimize for a maximum of four relations, because more relations exceeds the capabilities of our hardware for simulation. We found that the optimizer SPSA (maxiter=125) performed more accurately for both algorithms when compared to the optimizer COBYLA (maxiter=125) when used with any of the mentioned algorithms.

On comparing both algorithms, we found that the QAOA has better consistency and accuracy than the VQE algorithm. We have also been able to see the performance of both algorithms with the different optimizers and we found that although the SPSA performed more accurately than COBYLA, the time required for the latter is very less than the former. To execute 66 queries joining three relations with the QAOA algorithm, the COBYLA optimizer took 9.9 seconds while 79.24 seconds for the SPSA optimizer. For the same number of queries joining three relations with the VQE algorithm, COBYLA took 18.31 seconds and the SPSA optimizer 37 seconds. It is interesting to note that QAOA is extremely fast with COBYLA but very slow with SPSA when the algorithms are configured similarly, but VQE is a little slower with COBYLA when compared to QAOA+COBYLA, but it is faster with SPSA for the same comparison. We may examine the information for the four relational 185 queries to increase its certainty. The approximation approach took 720.25 seconds with COBYLA and SPSA (4599.4 seconds). Regarding the variational technique, it took 1465.4 and 3544.6 seconds, respectively, with COBYLA and SPSA.

5 SUMMARY AND CONCLUSIONS

We have explored join order optimization in this study by representing it as QUBO problem using Quantum Annealing, Simulated annealing, VQE, and QAOA algorithms. We have compared the performance of the QAOA and VQE algorithms with SPSA and COBYLA optimizers for the gate-based simulators by analyzing the percentage of optimal join order. Meanwhile, we also compared the performances of the D-Wave simulator and the D-Wave annealer by analyzing the percentage of optimal and valid shots obtained for each number of relations and we also analyzed the maximum, minimum and mean number of valid shots. Referring [37], we demonstrated better performance of our QUBO formulation on D-Wave annealer for up to five relations and we were able to produce valid join order trees for six relations. Additionally, we were able to retrieve optimal bushy join trees that are not supported by the contribution [37].

Our work has an advantage due to our QUBO formulation, which needs to be tuned more in order to see the potential of our formulation. We have verified that for 3 and 4 relations QAOA algorithm performs more accurately and consistently than the VQE algorithm. We also learned that the SPSA optimizer is a better choice than the COBYLA optimizer for both QAOA and VQE algorithms for our formulation when it comes to the accuracy of performance. But the optimization time of COBYLA is much smaller than the one of SPSA. We found that, for QUBO optimization, the qiskit simulators have very limited capabilities for queries joining a higher number of relations. In future work, we will further check the speedup of the runtime, analyzing higher penalties to undesired constraints and

experiment with the gate-based circuits by fine-tuning the QUBO formulation.

ACKNOWLEDGMENTS

This work is funded by the German Federal Ministry of Education and Research within the funding program quantum technologies - from basic research to market – contract number 13N16090.

REFERENCES

[1] 2023. Row estimation examples. https://www.postgresql.org/docs/current/row-estimation-examples.html
[2] Tameem Albash and Daniel A Lidar. 2018. Adiabatic quantum computation. *Reviews of Modern Physics* 90, 1 (2018), 015002.
[3] David Amaro, Carlo Modica, Matthias Rosenkranz, Mattia Fiorentini, Marcello Benedetti, and Michael Lubasch. 2022. Filtering variational quantum algorithms for combinatorial optimization. *Quantum Science and Technology* 7, 1 (2022), 015021.
[4] Rhonda Au-Yeung, Nicholas Chancellor, and Pascal Halffmann. 2022. NP-hard but no longer hard to solve? Using quantum computing to tackle optimization problems. *arXiv* 2212.10990 (2022). https://doi.org/10.48550/arxiv.2212.10990
[5] Dimitris Bertsimas and John Tsitsiklis. 1993. Simulated annealing. *Statistical science* 8, 1 (1993), 10–15.
[6] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. 2017. Quantum machine learning. *Nature* 549, 7671 (2017), 195–202.
[7] Olivier Carnal and Jürgen Mlynek. 1991. Young's double-slit experiment with atoms: A simple atom interferometer. *Physical review letters* 66, 21 (1991), 2689.
[8] Marco Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, et al. 2021. Variational quantum algorithms. *Nature Reviews Physics* 3, 9 (2021), 625–644.
[9] Pablo Díez-Valle, Diego Porras, and Juan José García-Ripoll. 2021. Quantum variational optimization: The role of entanglement and problem hardness. *Phys. Rev. A* 104 (Dec 2021), 062426. Issue 6. https://doi.org/10.1103/PhysRevA.104.062426
[10] Christoph Durr and Peter Hoyer. 1996. A quantum algorithm for finding the minimum. *arXiv preprint quant-ph/9607014* (1996).
[11] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028* (2014).
[12] Richard P Feynman. 2018. Simulating physics with computers. In *Feynman and computation*. CRC Press, 133–153.
[13] Aleta Berk Finnila, Maria A Gomez, C Sebenik, Catherine Stenson, and Jimmie D Doll. 1994. Quantum annealing: A new method for minimizing multidimensional functions. *Chemical physics letters* 219, 5-6 (1994), 343–348.
[14] Fred Glover, Gary Kochenberger, and Yu Du. 2018. A tutorial on formulating and using QUBO models. *arXiv preprint arXiv:1811.11538* (2018).
[15] Fred Glover, Gary Kochenberger, and Yu Du. 2019. A Tutorial on Formulating and Using QUBO Models. arXiv:1811.11538 [cs.DS]
[16] David J Griffiths and Darrell F Schroeter. 2018. *Introduction to quantum mechanics*. Cambridge university press.
[17] Lov K Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*. 212–219.
[18] Lov K Grover. 1997. Quantum mechanics helps in searching for a needle in a haystack. *Physical review letters* 79, 2 (1997), 325.
[19] Tadashi Kadowaki and Hidetoshi Nishimori. 1998. Quantum annealing in the transverse Ising model. *Physical Review E* 58, 5 (1998), 5355.
[20] Scott Kirkpatrick, C Daniel Gelatt Jr, and Mario P Vecchi. 1983. Optimization by simulated annealing. *science* 220, 4598 (1983), 671–680.
[21] Stephen Klassen. 2011. The photoelectric effect: Reconstructing the story for the physics classroom. *Science & Education* 20 (2011), 719–731.
[22] Wen-Yang Ku and J Christopher Beck. 2016. Mixed integer programming models for job shop scheduling: A computational analysis. *Computers & Operations Research* 73 (2016), 165–173.
[23] Mark W. Lewis and Fred W. Glover. 2017. Quadratic Unconstrained Binary Optimization Problem Preprocessing: Theory and Empirical Analysis. *CoRR* abs/1705.09844 (2017). arXiv:1705.09844 http://arxiv.org/abs/1705.09844
[24] Nana Li, Yujuan Liu, Yongfeng Dong, and Junhua Gu. 2008. Application of ant colony optimization algorithm to multi-join query optimization. In *Advances in Computation and Intelligence: Third International Symposium, ISICA 2008 Wuhan, China, December 19-21, 2008 Proceedings 3*. Springer, 189–197.
[25] Ryan Marcus and Olga Papaemmanouil. 2018. Deep reinforcement learning for join order enumeration. In *Proceedings of the First International Workshop on Exploiting Artificial Intelligence Techniques for Data Management*. 1–4.
[26] Sam McArdle, Tyson Jones, Suguru Endo, Ying Li, Simon C Benjamin, and Xiao Yuan. 2019. Variational ansatz-based quantum simulation of imaginary time evolution. *npj Quantum Information* 5, 1 (2019), 75.
[27] Xiao Mingyao and Li Xiongfei. 2015. Embedded database query optimization algorithm based on particle swarm optimization. In *2015 Seventh International Conference on Measuring Technology and Mechatronics Automation*. IEEE, 429–432.
[28] Florian Neukart, Gabriele Compostella, Christian Seidel, David Von Dollen, Sheir Yarkoni, and Bob Parney. 2017. Traffic flow optimization using a quantum annealer. *Frontiers in ICT* 4 (2017), 29.
[29] Michael A Nielsen and Isaac Chuang. 2002. Quantum computation and quantum information.
[30] Christos Papalitsas, Theodore Andronikos, Konstantinos Giannakis, Georgia Theocharopoulou, and Sofia Fanarioti. 2019. A QUBO model for the traveling salesman problem with time windows. *Algorithms* 12, 11 (2019), 224.
[31] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J Love, Alán Aspuru-Guzik, and Jeremy L O'brien. 2014. A variational eigenvalue solver on a photonic quantum processor. *Nature communications* 5, 1 (2014), 4213.
[32] Daniel E Platt. 1992. A modern analysis of the Stern–Gerlach experiment. *American Journal of Physics* 60, 4 (1992), 306–308.
[33] John Preskill. 2018. Quantum computing in the NISQ era and beyond. *Quantum* 2 (2018), 79.
[34] Jun John Sakurai and Eugene D Commins. 1995. Modern quantum mechanics, revised edition.
[35] Wolfgang Scheufele and Guido Moerkotte. 1996. *Constructing optimal bushy processing trees for join queries is np-hard*. Technical Report. https://pi3.informatik.uni-mannheim.de/~moer/Publications/MA-96-11.pdf Technical reports, Universität Mannheim.
[36] Maria Schuld, Ilya Sinayskiy, and Francesco Petruccione. 2015. An introduction to quantum machine learning. *Contemporary Physics* 56, 2 (2015), 172–185.
[37] Manuel Schönberger, Stefanie Scherzinger, and Wolfgang Mauerer. 2023. Ready to Leap (by Co-Design)? Join Order Optimisation on Quantum Hardware. In *Proceedings of ACM SIGMOD/PODS International Conference on Management of Data*.
[38] P Griffiths Selinger, Morton M Astrahan, Donald D Chamberlin, Raymond A Lorie, and Thomas G Price. 1979. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*. 23–34.
[39] Peter W Shor. 1994. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*. Ieee, 124–134.
[40] Michael Steinbrunn, Guido Moerkotte, and Alfons Kemper. 1997. Heuristic and randomized optimization for the join ordering problem. *The VLDB Journal* 6 (1997), 191–208.
[41] Arun Swami and Anoop Gupta. 1988. Optimization of large join queries. In *Proceedings of the 1988 ACM SIGMOD international conference on Management of data*. 8–17.
[42] Immanuel Trummer and Christoph Koch. 2015. Multiple query optimization on the D-Wave 2X adiabatic quantum computer. *arXiv preprint arXiv:1510.06437* (2015).
[43] Tobias Winker, Sven Groppe, Valter Uotila, Zhengtong Yan, Jiaheng Lu, Maja Franz, and Wolfgang Mauerer. 2023. Quantum Machine Learning: Foundation, New Techniques, and Opportunities for Database Research. In *Proceedings of ACM SIGMOD/PODS International Conference on Management of Data (SIGMOD)*. https://doi.org/10.1145/3555041.3589404
[44] Tobias Winker, Umut Çalikyilmaz, Le Gruenwald, and Sven Groppe. 2023. Quantum Machine Learning for Join Order Optimization using Variational Quantum Circuits. In *Proceedings of the International Workshop on Big Data in Emergent Distributed Environments (BiDEDE), Seattle, WA, USA*. https://doi.org/10.1145/3579142.3594299